Volt & Co — Can we defend this release?

Presenter delivery contract

Evidence-first revision, 13 November 2026. This script is for a 90-minute internal technical session: 80 minutes content and 10 minutes Q&A. All customers, products, orders and policy documents are synthetic.

Current readiness: the revised runtime, additive v2 tests and release workflow are local implementation. Azure v2 deployment, native evaluation, model-judge calibration, GitHub protection configuration and promotion have **not** been run or activated. The deployed historical DEMO v1 and its original nine-case results remain unchanged. Use the explicitly local routes below until new cloud receipts exist. This is not a claim that the revised end-to-end live route is rehearsed.

Four switch pairs: slide **7 → demo → 8**, **12 → demo → 13**, **15 → demo → 16**, **19 → demo → 20**. Display the checkpoint, speak the bridge, Alt+Tab to the prepared application group, stay there for the demonstration, then return and advance once. Never leave a slide unnoticed or switch to a new feature without explaining the question it answers.

Use the private display for these notes. Project only selected code lines, readable terminal output and prepared portal tabs; never environment values, connection headers or private Memory records. Use large fonts and rehearse browser zoom on the actual projector. Commands below run from the repository root.

The source-slide references were checked against the fresh local workshop copy supplied by the user: 198 slides, 35 hidden and 105 with speaker notes. The main review references match that copy. The online version was not independently inspected. No restricted AKS/A-B content is imported.

Say “recorded” when displaying a prior execution, and “local” when showing a test. A new invocation is live only when its new response is observed. Preserve errors. No new optimization, deployment, merge or recurring monitor is a stage action. The historical 8/9 → 9/9 repair was instructions-only; do not call it deterministic code enforcement. The local v2 contract is a separate revision.

Slide 01 — Can we defend this release?

Clock 00:00–02:00 · 2 minutes slides.

“Building an agent that answers a question is increasingly straightforward. Defending the answer, and defending the decision to release that agent, is the harder engineering problem.

“Today we will follow a single Volt & Co support request. A customer wants a replacement charger. The agent needs business facts, policy evidence and a clear boundary around what it can actually do.

“Our question is not simply, ‘Does it sound helpful?’ It is, ‘Can we explain the result, identify what was tested and reopen the evidence behind the release?’

“Microsoft Foundry is the agent platform. Citadel gives us a governance architecture around selected model and tool routes. GitHub Copilot helps us develop the application. We will keep those responsibilities distinct.

“You will also see a genuine limitation: an answer that passed the original business checks still claimed an action the system could not perform. That is the most useful moment in the session, because it shows why operation must teach development.

“The customer and data are fictional. Some evidence was captured before today; I will identify it. The revised contract is currently locally implemented, with cloud activation still awaiting approval. We will never make a planned control look like a completed one.”

[Show the question, not a feature agenda. At 02:00 advance to slide 2.]

Slide 02 — One agent. Two loops. Four demonstrations.

Clock 02:00–05:00 · 3 minutes slides.

“This is the route. First we build and inspect one request. Next we prove what the behavior means. Then we inspect the release gate and its case file. Finally, an operational finding comes back as a test and a repair.

“Tools, knowledge, evaluation, guardrails and tracing appear where the request needs them. They are not separate product tours.

“The four demo cards mark exactly when we leave the slides and which slide we return to. Inside a demonstration, I will stay with the application and evidence rather than bounce back to a feature slide every minute.

“There are also two different kinds of evidence. A live request tells us what happened now. A recorded, versioned run lets us inspect what happened before and compare it consistently. Neither is a substitute for the other.

“The original release evidence is real and complete for its original tests. The new runtime and automated workflow are separate work: we can inspect and test their implementation locally, but we need new cloud results before calling them deployed and validated.

“We have ten minutes reserved for questions at the end. I will take a short clarifying question when it blocks understanding, but keep broader architecture discussions for that slot.”

[Briefly point out the four numbered demos. Do not read every feature label. Advance at 05:00.]

Slide 03 — Two loops. One improving service.

Clock 05:00–10:00 · 5 minutes slides.

“The inner loop is where a developer makes and proves a change. We connect a model, tools and policy; run a request; inspect the execution; evaluate it; and improve the application.

“The outer loop makes that work repeatable. A candidate carries evidence into a release gate. The approved artifact reaches a serving environment, and we observe its behavior.

“The top bridge is a handoff, not a promise. It carries an identifiable candidate and a case file: source, instructions, package digest, capability bindings, dataset and evaluator versions, and the results.

“The bottom bridge is learning. An operational trace is evidence of what happened. It is not automatically an example of what should happen. A person reviews the finding and supplies the expected behavior before it becomes a regression.

“This is why one agent and one scenario matter. If we used a shopping agent for tools, a finance agent for evaluations and an unrelated chatbot for monitoring, we would show features but not establish the bridges between them.

“Our serving environment is called DEMO because it is synthetic and production-like, not customer production. DEV and DEMO share an account in this cost-controlled profile. The labels alone are not a hard production isolation boundary.

“Here is a useful question to keep in mind. If someone changes an instruction tomorrow, can we identify which package is serving, which tool set it used and which tests support that release? If not, the missing piece might not be a better model. It might be one of these bridges.

“We will revisit this lifecycle on each demo divider. Build, prove, release, operate and improve are different views of the same system.”

[Point to each bridge and the shared contract. Invite one brief check of understanding; do not turn this into a survey of CI/CD tools. At 10:00 advance.]

Slide 04 — One customer. One accountable answer.

Clock 10:00–11:00 · 1 minute slides.

“Our customer has order VC-10482, a Volt Charge 65 and batch CH-0722. They are outside the standard return window and ask whether they can be assessed for a replacement.

“The order, batch recall and replacement eligibility are business facts. The policy explains the assessment path. The agent should combine them into a useful next step.

“Eligible for assessment does not mean approved replacement, shipped stock or an issued refund. We have no transaction or human-handoff integration. That distinction will matter later.

“The same request, its evidence and its versions will remain our thread through the entire session.”

[Advance to the runtime choice. No terminal yet.]

Slide 05 — The harness runs the loop. We own the contract.

Clock 11:00–13:00 · 2 minutes slides.

“The harness manages the agent's execution loop: sending context to the model, handling tool calls and producing a response. The application still owns the meaning of its business contract.

“Start with a managed configuration path when it meets the need. Hosted is appropriate when we deliberately need application-owned behavior and are prepared to maintain its code and dependencies.

“Our implementation uses Python Agent Framework and the Responses protocol. The new business path derives the final decision from matching tool evidence and controls customer-facing wording before it leaves the response stream. That is the code-level reason for this hosted choice.

“Be precise about chronology. The earlier improvement changed instructions and passed the original tests. It was not deterministic enforcement. The new evidence contract is a separate local revision, not something we retroactively claim existed in the first deployment.

“Copilot is the development assistant here. It can help author the tests and repair, but its proposal has to satisfy the same contract. It is not the serving runtime.

“Alternative managed-harness experiences are useful options, not a reason to run a second implementation track in this session.”

[Show main.py only later inside Demo 1. Do not claim parity between agent types or quote internal preview dates.]

Slide 06 — Where Citadel and Foundry meet

Clock 13:00–15:00 · 2 minutes slides.

“This is the architecture around our one request. Foundry hosts the agent and provides the tool, knowledge, evaluation and observability surfaces. Citadel's Governance Hub mediates the model and business-tool routes we have configured through API Management.

“The gateway gives us a place for access contracts and consumption controls. It does not automatically govern every internal evaluation, Memory or IQ model call, and a token quota is not a universal spending cap.

“We also separate identities by responsibility. The platform caller, hosted runtime and release operator are not the same actor. The new automated release path proposes separate OIDC identities for DEV and DEMO; those permissions still need approved activation.

“The broader source diagram includes architecture context. Its presence does not mean every security and management integration is enabled here.

“For this profile, Foundry is public and Entra-authenticated; backend storage and Key Vault remain private. Resource location is not a Sweden-only processing guarantee, and APIM Developer is not a production SLA choice.

“Now let us make the diagram concrete by running and inspecting the customer journey.”

[At 15:00 advance to the Demo 1 card.]

Slide 07 — Demo 1 · Build and inspect

Clock 15:00–25:00 · 9 minutes demonstration + 1 minute transitions.

[SWITCH OUT — show the card, say the bridge, then Alt+Tab to the prepared terminal. This command targets the unchanged historical DEMO v1, not the new local code.]

“Let us ask the released agent the replacement question. Watch for the facts it used, the policy it cited and what it says it actually did.”

```
$env:PYTHONIOENCODING = "utf-8"
$python = ".\agent\src\.venv\Scripts\python.exe"
$out = ".\.runtime\live-" + (Get-Date -Format "yyyyMMdd-HHmmss")
& $python .\scripts\capture_cases.py `
--project-endpoint "https://fdy-volt-citadel-261111.services.ai.azure.com/api/projects/volt-demo" `
--expected-version 1 --dataset .\agent\src\.foundry\datasets\volt-regression.jsonl `
--case replacement-recall --fresh --out $out --pause-seconds 0
```

[15:15–17:30: run once only during an authorized rehearsal/presentation. If pending after 30 seconds, announce the recorded fallback and open Recorded evidence.html → Released DEMO v1 → replacement-recall. Preserve the actual new result. The current implementation session does not run this usage-billed command.]

“This command requires a new output directory and refuses recorded-response reuse. If I switch to our saved capture, I will say so.

“Look at the answer and structured decision separately. The next step is assessment advice. Now compare the order and batch against the tool arguments and results. A tool name in a catalog is configuration; a matched call and output is execution evidence.”

[17:30–20:00: inspect pre-opened DEMO Toolbox v1 and the captured order, recall and eligibility calls. DEV Toolbox v2 is a different binding. Show the two synthetic policy references, then the actual source documents in Knowledge or knowledge/replacement-policy.md and knowledge/recall-notice.md.]

“The Toolbox curates the capabilities for this agent. The backend supports more operations, but this journey exposes three business operations and one IQ tool. A smaller capability set is deliberate.

“IQ makes policy evidence retrievable and inspectable. A citation still has to support the statement; it does not authenticate ownership of an order or change the eligibility result.

“Now the trace. We want to see the model and tool sequence as a developer, not wait until an operations incident to use observability.”

[20:00–22:30: use a freshly correlated trace only if available for the exact new response. Otherwise open the identified saved golden trace in the offline viewer and say it is a separate recorded request. Its response is caresp_03f2cedafe9a81ff00ml4CEH7VZPqFrxR1SFFHpglDzRb3u45a and it has 43 exported spans. Do not claim that those spans belong to today's invocation.]

“This trace identifies one real execution and where work occurred. It is not an all-traffic latency benchmark. The useful engineering property is that we can connect behavior to its actual path and version.

“Finally, two code regions explain the ownership boundary. Agent Framework runs the loop. Our application decides what business output is allowed to leave it.”

[22:30–24:30: show agent/src/main.py create_support_agent and evidence_contract.py supported_response/process. Explain that this is local v2 source, not serving v1. The renderer buffers model text, retains actual tool events and derives a constrained answer. No repository tour.]

“The new renderer is stricter than an instruction. It can reject missing or conflicting evidence and suppress unsupported model wording before the customer sees it. The cloud deployment of this revision is still pending; the next demo shows the local proof rather than claiming it is already serving.”

[RETURN — at 24:30 Alt+Tab back to slide 7 and advance once to slide 8. Finish by 25:00.]

If the answer says “I'm escalating”: “No ticket or handoff was created. That is an unsupported action claim, not merely a stylistic issue. We will inspect that exact gap in the operating chapter.”

If a tool fails: “The evidence is incomplete. We preserve the failure rather than borrow a passing answer from another run.”

Slide 08 — A useful answer is only the beginning.

Clock 25:00–26:00 · 1 minute slides.

“Back from the first demo, we can explain the capabilities and the execution path. That is an important foundation, but it does not establish that the final answer preserves the business meaning.

“A well-grounded answer can still choose the wrong structured decision. It can also describe an action that no tool performed.

“We therefore need more than a single quality score. Let us separate what code can check, what a semantic judge can assess and what a human must decide.”

[Advance to slide 9; remain in PowerPoint until slide 12.]

Slide 09 — Three ways to judge the same behavior

Clock 26:00–28:00 · 2 minutes slides.

“Code-based evaluation is strongest when we can state an objective contract. For this request, the reviewed expectation, the actual tool status and the final decision should agree. The call must also belong to the right order and batch.

“We do not replace the reviewed expectation with whatever the tool happened to return. A broken tool and a broken answer could agree. We need both the domain expectation and the observed evidence.

“The model-based rubric addresses a different question: does the prose claim an action that never happened? Advice to contact support, an offer to draft a message and a claim that a ticket was opened mean different things. A keyword check alone cannot reliably separate them.

“A human reviews the expected behavior and resolves ambiguous findings. We can use model judges at scale without pretending that they create business truth.

“Our v2 rubric has positive and negative calibration examples. Those examples are authored, but cloud calibration is not yet run. An uncalibrated judge cannot be presented as a verified release control.”

[Point to each responsibility. Keep exact thresholds and schema details in the demo/code.]

Slide 10 — Name the boundary. Show its evidence.

Clock 28:00–30:00 · 2 minutes slides.

"Authorization, content safety and business correctness answer different questions. Authorization determines who may reach a capability. A content guardrail checks supported content boundaries. The business contract determines whether the result means the right thing.

"We have a recorded native input-marker block and permitted routed requests. Those are actual proofs for those specific boundaries. A refusal sentence alone would not demonstrate platform enforcement.

"Tool-call controls and a bounded red-team exercise would strengthen the story, but they require support verification and an approved run on this actual route. We do not turn a selector in the source deck into a working control in our environment.

"Similarly, the new response contract is not universal safety. It is a precise application rule for this synthetic support path. Naming the boundary makes the claim narrower and more useful.

"The next decision is whether configuration optimization offers an improvement within those constraints."

[No new red-team job or guardrail assignment on stage. Advance.]

Slide 11 — Optimization is an experiment, not a promise.

Clock 30:00–31:00 · 1 minute slides.

“The completed Optimizer experiment used an instruction-only search and separate tuning and validation prompts. The result retained the baseline at zero point seven five; no optimized candidate was accepted or applied.

“That is evidence too. Succeeded describes the job, not an improvement. The two-candidate setting was a budget, not proof that two candidates completed.

“We will use the completed result. It took more than eighteen minutes, so waiting for another search would be a poor use of this session. A code contract and an optimization objective are separate responsibilities.”

[Advance to Demo 2 at 31:00.]

Slide 12 — Demo 2 · Prove the contract

Clock 31:00–40:00 · 8 minutes demonstration + 1 minute transitions.

[SWITCH OUT — open the completed native baseline report from Rehearsal bindings.json. Do not submit a native evaluation in this local-only revision.]

“Let us inspect a failed item, not just a green percentage. The original hosted baseline passed eight of nine business cases. The eligibility tool returned eligible while the final structured decision said recommendation.”

[31:15–33:00: show baseline failed replacement-recall item and the candidate's completed 9/9 report. State that the original fix changed instructions only. Native grading scored captured responses; opening the report does not rerun the agent.]

“The historical instructions change improved those observed responses. That does not make the decision code-enforced. We now have a separate implementation that must pass stronger tests.

“A useful negative test makes the model try to return the wrong decision and claim an escalation. The test should fail if that text reaches the customer.”

```
& .\agent\src\.venv\Scripts\python.exe -m pytest `
.\agent\src\tests\test_evidence_contract.py `
.\evaluations\test_business_contract_v2.py -q
```

[33:00–36:30: run the local suite. Show the test with ScriptedClient, actual Agent tool execution and the local Responses HTTP stream. These are deterministic local tests with scripted model output—not an Azure model benchmark. Read test outcomes only after completion.]

“This is the actual Agent construction and host adapter path. We are not testing an unused helper. The scripted model emits an unsupported answer; the response boundary instead emits the tool-derived decision and explicit no-action wording.

“We also test missing, mismatched and conflicting evidence. The expected behavior is an explicit unavailable result, not a convenient eligible default.

“V1 and its scores are preserved. V2 adds checks instead of silently changing the meaning of an old nine-of-nine label.”

[36:30–39:30: inspect the eight calibration examples in evaluations/action_truthfulness_calibration.jsonl. Then show the actual completed Optimizer no-improvement panel in the offline viewer or portal. If time is short, skip token counters and capacity history.]

“These examples distinguish advice, inability, a draft offer and an executed handoff claim. The model rubric must demonstrate that distinction in a separately approved calibration run before it can gate a release.

“The Optimizer result is different evidence: no accepted gain. We do not confuse its task-adherence score with business correctness or credit it for the code revision.”

[RETURN — at 39:30 return to slide 12 and advance to slide 13.]

If a local test fails: “The new contract has not passed this check. We inspect the failure; we do not describe the candidate as releasable.”

Slide 13 — Bridge 1 · Candidate + evidence + approval

Clock 40:00–42:00 · 2 minutes slides.

“Back from the tests, we have a candidate and evidence about its behavior. Bridge one makes that evidence travel with the exact artifact.

“The package is built once. Its commit and digest remain associated with the agent version, capabilities, dataset and evaluator versions. Rebuilding after evaluation would produce a different artifact that needs its own proof.

“The automated path has three boundaries: trusted code enters an approved DEV evaluation, the exact commit receives its result, and a separate approval controls DEMO promotion.

“The workflow file alone does not establish that protection exists. The implementation checks actual repository and environment settings before it acquires cloud credentials. Those settings are not activated yet.

“Our prior release was operator-run. We preserve that history and its receipts. The new automated gate will have its own runs and case files; it must not borrow the old release's success.”

[Advance to the case-file slide.]

Slide 14 — A case file you can reopen

Clock 42:00–43:00 · 1 minute slides.

“A green badge is a verdict. The case file explains why the verdict was reached.

“It should identify the exact source and package, the candidate and serving versions, the dataset and evaluator, and each relevant outcome. It should include failures as well as successes.

“For our new gate that also includes calibration and action-truthfulness judgments. Approval and post-promotion verification are separate records.

“If another engineer cannot reopen that chain, the badge is doing too much work in the story.”

[At 43:00 advance to Demo 3.]

Slide 15 — Demo 3 · Inspect the release gate

Clock 43:00–50:00 · 6 minutes demonstration + 1 minute transitions.

[SWITCH OUT — open .github/workflows/behavioral-release.yml and docs/RELEASE-GATE.md in the prepared editor. Current route is implemented code inspection plus local checks; no new cloud pass exists.]

"Let us look at the release mechanism, and be explicit about its readiness. This workflow is implemented but not activated. I will show the actual checks rather than invent a successful run."

[43:15–45:00: show workflow_dispatch inputs, exact commit, trusted same-repository PR, serialized concurrency and the two environment names. Then show live_gate_preflight.py requiring real reviewers, disabled self-approval/bypass and the required main-branch status.]

"The operator selects a reviewed commit. The preflight rejects a fork, draft or changed head. It also checks actual approval controls; a friendly environment name is not an approval gate.

"Cloud access uses distinct OIDC identities. We are not copying a developer credential into CI. Their exact subjects and roles still require approved configuration.

"The evaluation stage packages once, deploys to DEV, captures fresh pinned responses, runs native v2 grading and calibrates the semantic action judge. A missing case or errored judge cannot produce a passing summary."

```
& .\agent\src\.venv\Scripts\python.exe -m pytest `
.\evaluations\test_capture_gate.py .\evaluations\test_live_gate.py -q
```

[45:00–47:00: run local gate tests, especially rejection of stale green checks, missing case evidence and unconfigured environment protection. These are local contract checks, not a configured GitHub environment or live deployment.]

"Promotion then checks the merged exact PR and evaluates the same response identities across the native and semantic gates. It uploads the saved ZIP without rebuilding and performs independent DEMO validation.

"One defect identified in the review was historical branch and check-name pinning. That was valid for the earlier feature-branch release, but not a reusable main-targeted gate. The revised checks use the current target and reject a newer failed rerun rather than accepting an older green one."

[47:00–49:30: inspect existing .runtime/volt-candidate-package.json and .runtime/volt-demo-release.json only as HISTORICAL V1 receipts. Show the same digest and native report association. Do not execute promote_demo.py or relabel those receipts as v2 workflow output.]

"Here is what a real historical case file already establishes: source identity, the same ZIP, native results and verified DEMO behavior. What it does not establish is that the newly written workflow executed. That distinction is deliberate.

"Once the new cloud trial is approved, this same walkthrough opens its real Actions run and downloadable case file. Until then, the readiness boundary stays visible."

[RETURN — at 49:30 return to slide 15 and advance to slide 16.]

If asked to run the workflow now: "That needs separately approved usage, OIDC permissions and protection configuration. The stage is not the place to broaden access or bypass a gate."

Slide 16 — The release passed. The contract missed something.

Clock 50:00–54:00 · 4 minutes slides and recorded evidence discussion.

“Back from the release gate, here is the operational surprise. This is not the old DEV eligibility failure. It is a real answer from the released synthetic DEMO version.

“The answer says, ‘I'm escalating this to human support.’ But there is no handoff tool. The tools only read order, recall and eligibility facts.

“The structured eligible decision is correct for assessment. The policy references are present. Those were the things the original grader checked, so this answer still passed.

“That does not invalidate the historical nine-of-nine result. It tells us what the result meant—and what it did not cover. The contract was missing a semantic action-truthfulness criterion.

“We should not quietly rewrite the recording to make the answer look safer. We preserve the exact text and version. Nor should we replay an old DEV failure and call it a production finding.

“This one really is outer-loop evidence: it came from the released DEMO response. It has a matching saved trace, and it identifies a gap we can turn into an explicit expectation.

“For a real customer, this matters. A claim that support has already been notified can change what the customer does next. The action boundary is not just a wording preference.”

[Point to 9/9 and the quote. Allow one short audience reaction: the lesson is metric scope, not that tests are worthless. Advance at 54:00.]

Slide 17 — Observe the request, not just the aggregate.

Clock 54:00–57:00 · 3 minutes slides.

“The finding has an execution identity. The same response ID occurs in the saved answer and the exported App Insights trace. That lets us inspect what actually happened instead of relying on an incident summary.

“There is a different historical DEV request whose correlation identifier was not found in exported telemetry. We keep that gap explicit. It does not remove the verified correlation for this later DEMO answer.

“A production monitoring design would sample traffic, evaluate relevant quality and safety dimensions, and lead an operator back to the exact response. That is the purpose of continuous evaluation.

“We have not configured a recurring monitor or native Insights scan in this environment. A manual repository annotation is not a native Insights finding, and a scheduled GitHub batch is not automatically the same as native continuous evaluation.

“The local core revision preserves the real finding and prepares its regression. Recurring sampling, retention, cost limits and ownership need a separate approved operating decision.

“What matters for the feedback bridge is that the incident is real, its provenance is preserved and the expected behavior is reviewed before we turn it into a test.”

[Use the stage marker. Do not navigate to an empty monitoring page and imply that it is enabled.]

Slide 18 — Bridge 2 · A finding becomes a test

Clock 57:00–60:00 · 3 minutes slides.

“The reviewed expectation is simple: recommend that the customer contacts support, but do not claim a ticket or handoff was performed.

“That expectation becomes an additive escalation case and an action-truthfulness rubric. We keep the original cases unchanged so we can detect a repair that breaks an ordinary, missing or unknown-input path.

“The historical capture should fail the new criterion. That is not a new invocation of the old agent; it is a transparent regrading of preserved evidence under a new contract.

“Then the application needs a real repair. Instructions remain useful, but they are not the only line of defense. The new business renderer derives the decision from matching results and generates constrained wording for the supported outcomes.

“Copilot can help implement this, but the acceptance criterion belongs to the reviewed test and gate, not to the assistant's explanation.

“This is the second bridge: observed behavior, reviewed expectation, a test, a change and new release evidence. We will inspect that sequence now, with cloud-dependent steps clearly separated from what has passed locally.”

[At 60:00 show the final demo checkpoint.]

Slide 19 — Demo 4 · Turn the finding into a repair

Clock 60:00–70:00 · 8 minutes 30 seconds demonstration + 1 minute 30 seconds transitions.

[SWITCH OUT — open recordings/escalation-finding.json, recordings/demo-v1.json and the saved trace view. Show only the synthetic answer and selected identifiers, not broad raw telemetry.]

"Here is the recorded DEMO claim, its response identity and the matching trace. The repository annotation states the expected boundary. It is based on the reviewed finding, not generated proof that an Insights service ran."

[60:30–62:00: compare response_id in the finding, demo-v1 capture and demo-trace export. Show the exact answer unchanged.]

"Now the test. The original grader returns one for this answer. The new criterion returns zero. This demonstrates the escaped gap without staging a new failure or pretending a different model answer was the old incident."

```
& .\agent\src\.venv\Scripts\python.exe -m pytest `
.\evaluations\test_business_contract_v2.py `
.\agent\src\tests\test_evidence_contract.py -q
```

[62:00–65:00: show the exact regression test and run it. Then open the additive escalation-request case in volt-regression-v2.jsonl. Explain why expected outcomes remain alongside actual tool evidence.]

"The new case varies the second customer moment: it asks for escalation and an honest account of what was actually done. It is not a second run of the same replacement prompt.

"Here is the repair path. The model may still emit unsupported wording internally, but the business response boundary buffers it. Only a response derived from valid matching tool evidence reaches the customer.

"The local HTTP test exercises the real Responses host and checks the stream. This matters because filtering only a final result would be too late if the claim had already streamed."

[65:00–67:00: inspect the focused evidence_contract.py diff and main.py registration. Highlight the valid-evidence branch, logged unavailable branch and stream buffering. Show that Memory is a separate DEV-only exercise, outside this business renderer.]

"We also test another order, duplicated or conflicting results, missing identifiers and incomplete tool execution. None should quietly become a successful eligible response.

"The deterministic phrase regression and model rubric are complementary. The phrase check catches this known incident locally. The semantic judge still needs a calibrated Azure run to handle varied wording reliably."

[67:00–68:30: show the authored action rubric and gate requirements. Current route ends at local fail/fix/pass. Do not invoke old DEMO v1 and describe it as the repaired release. After a separately approved v2 deployment and native/model gate, replace this checkpoint with a fresh escalation-request against its actual recorded version, using --fresh and the v2 dataset.]

"The local repair is now a candidate, not an already released service. Its next step is the approved cloud trial and the same evidence-bearing gate we inspected. We retain the earlier receipts and create new ones; we do not edit history to make the story appear complete.

"That is the operating discipline: the finding changes our test contract, the test constrains the repair, and the gate decides when the repair may serve users."

[RETURN — at 68:30 return to slide 19 and advance once to slide 20 by 70:00. Do not consume the closing or Q&A budget waiting for a job.]

If a test fails: "The candidate has not met the new expectation. This is exactly why the gate exists." Preserve and investigate it; do not substitute the historical 9/9.

After cloud activation only: use a newly captured escalation-focused response and its own trace/version. A prior golden trace is not proof of this new request.

Slide 20 — The architecture decisions we can explain

Clock 70:00–74:00 · 4 minutes slides.

“Back from the feedback loop, these are the decisions we can explain rather than merely list.

“The application uses hosted Responses because we own the business response boundary. It uses read-only tools for authoritative facts and a constrained output contract. That choice brings code and dependency responsibilities.

“The platform provides hosting and lifecycle surfaces. The Toolbox curates capabilities; IQ supplies inspectable policy evidence; the governance route makes selected model and tool access explicit.

“The lifecycle makes evidence travel with a candidate. Evaluators answer different questions, the case file preserves their outcomes, and the feedback path turns a relevant operational finding into a reviewed test.

“Different identities own different responsibilities. A project split, a managed identity badge or a green workflow is not a substitute for checking the actual scope and enforcement.

“We also made choices about what not to include in the main route. Memory is useful, but it is disabled in DEMO and does not explain the replacement decision. It belongs in backup. Another agent type or integration channel can be discussed without becoming a second demo project.

“The design should be understandable enough that a customer team can choose a different component deliberately and still preserve the contract and evidence chain.”

[Use up to two minutes here for one architecture question or recover earlier delay. At 74:00 advance; do not repeat every caveat.]

Slide 21 — Your next POC: one decision, one evidence chain.

Clock 74:00–78:00 · 4 minutes slides.

“A useful next step is a scoped proof, not a commitment to deploy every capability we have discussed.

“Choose one business decision and a person who can judge it. Identify what the agent may read, what it may change and which actions require a human or another system.

“Define representative cases, including missing information and the claims the agent must never make without evidence. Decide what constitutes a release blocker before looking at a promising score.

“Then choose one evidence chain you will be able to reopen: the request, capabilities, expected outcome, candidate, test run, approval and serving version.

“Plan the operating feedback too. Who reviews findings? Which traffic may be sampled? How are privacy, retention and costs controlled? How does an approved finding become a regression?

“For our workshop, the remaining activation work is explicit: the new cloud run, semantic calibration and repository protection controls. We do not hide those prerequisites behind an impressive diagram.

“For a customer POC, success should similarly mean that the team can explain both what works and the boundary of the proof. That is a stronger outcome than an isolated chat that looked good once.”

[This is also overrun recovery. If behind, use only the first three paragraphs and move to takeaways. Q&A starts at 80:00.]

Slide 22 — Useful. Defensible. Improving.

Clock 78:00–80:00 · 2 minutes slides. Then 80:00–90:00 Q&A.

“Three lines summarize the session.

“The harness runs the loop; our application owns the business contract. Tools and policy make an answer useful, but the output must preserve what those facts actually establish.

“A green badge is a verdict; the case file is the evidence behind it. Source, artifact, tests, approvals and serving identity have to refer to the same change.

“Reviewed operational findings become tests before they become fixes. Our unsupported escalation claim showed why an original passing suite can still need to learn.

“That is the full value of the two loops. Foundry supplies the agent platform; Citadel supplies an architectural governance pattern; Copilot helps development; evidence connects the work.

“We will use the remaining ten minutes for your questions.”

[Stay on this slide. Use 23–24 only when a question calls for them.]

Slide 23 — Memory is a separate, narrow experiment.

[Hidden Q&A backup; no main-route time allocated.]

“Memory remembers a narrow synthetic response-style preference, not policy, identity or authority. The verified historical DEV experiment used four separate conversations: remember and read the native item, recall, forget and verify deletion, then confirm absence.

“That proves one authenticated caller's lifecycle. A new conversation is not a second identity, so cross-user isolation remains unproved.

“The preference has a seven-day retention setting. Forgetting the Memory item does not erase historical conversations or telemetry. The business DEMO keeps Memory disabled and does not receive the broader account roles used for that DEV experiment.”

[Never project raw Memory IDs or caller scope. The historical v5 session and preview metadata warning are documented in the operational guide.]

Slide 24 — Identity follows responsibility.

[Hidden Q&A backup; no main-route time allocated.]

“The caller, runtime and deployment identity have different jobs. The caller authenticates to the platform; the runtime reaches its approved project capabilities; the release identity needs only the rights for the explicitly approved deployment and evidence checks.

“Not all downstream hops are keyless. Toolbox-to-APIM and APIM-to-Functions use centrally managed scoped credentials. Those are not customer order-ownership checks.

“The shared DEV/DEMO account is a workshop tradeoff, not a hard production separation. A production design needs its effective scopes, networking, data processing and retention reviewed against the customer's requirements.”

[Use docs/OPERATIONS.md and docs/RELEASE-GATE.md for exact boundaries. Do not broaden roles during Q&A.]

Appendix A — Short answers

Was the original fix code-enforced? No. It changed instructions. The new local evidence renderer is the first code-enforced business response boundary in this project.

Why keep reviewed expected values when we have tool outputs? A faulty tool and an answer can agree. Check the expected business outcome and request-bound observed evidence independently.

Does 9/9 mean all answers are safe? No. It means nine cases passed that version of the metric. The real unsupported escalation claim demonstrates the missing semantic criterion.

Is the new automated gate active? No. Its implementation and local tests are available; OIDC, protection rules, native v2 evaluation and cloud model calibration need separate approval and execution.

Is this native Insights? The current incident is a reviewed repository annotation linked to a real response and trace. No native Insights monitor or portal annotation is configured.

Why did Optimizer succeed without improving? The job completed with stalled candidate screening and retained its baseline. No improved candidate was accepted or applied.

Will buffering affect latency? It delays first customer-facing text until the contract is checked. We have not measured the new hosted latency and make no performance promise.

Can it issue refunds or open tickets? No. All business tools are read-only. Adding mutations requires explicit authorization, ownership checks, idempotency, approvals and new tests.

Appendix B — Rehearsal and contingency

- Reconcile selected source slides by title against the linked online workshop after sign-in. Do not assume the downloaded copy has the same numbering.
- Use four demo entry/return pairs: 7/8, 12/13, 15/16, 19/20.
- Keep the main branch's serving historical evidence separate from locally implemented v2.
- Pre-open the current deck, source editor, terminal, native reports, offline viewer and relevant Foundry tabs.
- Test projector readability, sign-in and audio. Close unrelated tabs and notifications.
- Preserve v1 recordings and the immutable ZIP. Never rebuild them and claim the hashes stayed the same.
- Announce recorded fallback after a 30-second live-call delay; never submit duplicate traffic just to fill stage time.
- Use local tests for the pre-activation version of Demos 2–4. Replace pending checkpoints only after actual cloud evidence is captured and reviewed.
- Keep short privacy/preview/scope statements in the main route; use backup for detail.
- If five minutes behind, skip Optimizer counters, broad trace navigation and the second architecture question. Protect the incident, tested boundary and Q&A.
- Do not start a new optimization, recurring monitor, permission change, merge or promotion on stage.
- Complete a full human 90-minute rehearsal. Word estimates are not timing proof.